Predicting Ligue 1 Match Outcomes with Player-Level Statistics

Data Collection & Feature Organisation

Sienna O'Shea, Samuel Majbruch, Ioana Olaru & Ava Sudit

Ligue 1 · Seasons 2018–2024 · 1,653 matches · 73 features

Outline

Central Hypothesis

Research Question

Do individual football player rolling statistics add predictive signals beyond team-level aggregates?

Goal: predict Ligue 1 match outcomes (home win, draw, away win) using team and player data. Train on 2018–2022; test on 2023/24.

Three feature regimes evaluated:

Regime A

Team-level aggregates

29 features

Regime B

Team + FBref match statistics

50 features

Regime C

Full set incl. player-rolling features

76 features

Data Sources

FBref

Individual player stats per match and team-level summaries

62,683 player-match rows

Seasons 2018–2023

Understat

Match results, expected goals (xG), and pre-match win probabilities

2,105 matches

Seasons 2018–2023

Transfermarkt

Squad market values and average player age – an estimate for squad quality

174 team-seasons

Seasons 2018–2023

FBref data was collected via `soccerdata`; Understat via `understat`; Transfermarkt from an open-source GitHub repository.

Pipeline & Dataset

How we built it:

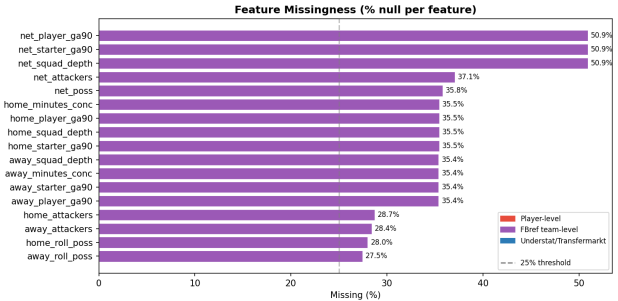
1. Match team names across data sources
2. Reshape each match into one row per team, ensuring home and away sides are handled symmetrically
3. Construct a 5-match rolling window using only prior games, so the current match never contributes to its own features
4. For Regime C, compute per-90 player statistics and average them across the starting eleven, weighted by minutes played
5. Remove the first 439 matches because they do not yet have enough prior history for rolling features

Output datasets:

Regime	Matches	Features
A: team only	2,092	29
B: +FBref	2,092	50
C: +player	1,653	76

Regime C is smaller: the first 439 matches have no prior rolling history.

Missing Data: Sources & Resolution



What is missing?

- No FBref *team* data for 2021/22 – not able to collect team-level data, but player-level was captured
- Rolling features need prior matches — first ≈5 matchdays per season are null (cold-start)
- `net_*` at ∼51%: null if *either* side null

What we did

- No imputation – null rows dropped at fit time
- Same subset used across all regimes
- Regime C: 439 cold-start rows dropped → 1,653 matches

What this means

- 2021/22 excluded equally from A, B, C – no regime penalised
- Regime C subset smaller – note when comparing across regimes
- Understat / Transfermarkt: zero missingness

Feature Engineering

Three feature layers:

Source	Features engineered	Regimes
Understat + TM	xG, goals, wins, points; value, age	A, B, C
FBref team	Possession, goals + assists per 90	B, C
Player	Shots, shots on target, tackles + interceptions, goals, assists per 90	C

Player aggregation (Regime C):

$$\hat{s}_{\text{team}} = \frac{\sum_{p \in \text{XI}} \text{min}_p \cdot \text{roll}_p}{\sum_{p \in \text{XI}} \text{min}_p}$$

Stats tracked per player

Feature	Meaning
`sh_p90`	shots per 90 min
`sot_p90`	shots on target per 90
`def_p90`	tackles + interceptions per 90
`gls_p90`	goals per 90 min
`ast_p90`	assists per 90 min
`starter_mins`	average minutes per game
`pct_fw`	share of XI minutes to forwards
`depth_sh`	shot gap: starters vs substitutes

- Compute player rolling stats on a per-90 basis.
- Aggregate the starting XI by minutes played.
- Produce home, away, and net versions of each team stat.

No data leakage: only the five matches before the current game are used.

Form follows the player: recent player form travels with transfers, so predictions rely on player form more than just current team affiliation.

Feature Selection — Four-Method Protocol

1. Filter Method

Simple idea: if I know this feature, do I get a better clue about the match result? Good for catching patterns that are not just straight lines.

2. Wrapper Method

Start with many features, remove the weakest ones step by step, and keep the set that gives the best validation score. Best subset: **26 features**.

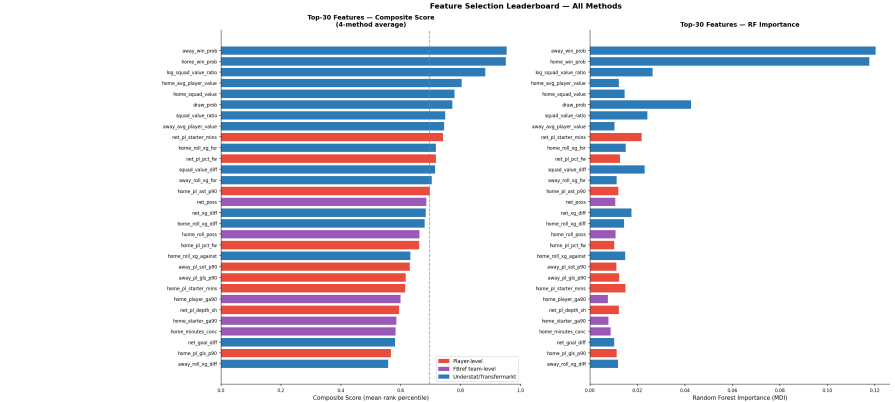
3. Embedded Method — Lasso

A linear model with a penalty that pushes unhelpful features down to exactly zero. 17 features were removed; **0 player features** were.

4. Embedded Method — Decision Tree

Let a forest of trees rank which features it uses most to make good splits. This gives a second opinion that broadly matches RFECV.

Feature Leaderboard: Top 30 Features



■ Understat/Transfermarkt ■ FBref team-level ■ Player-level

Feature Distributions by Match Outcome (Top 20)

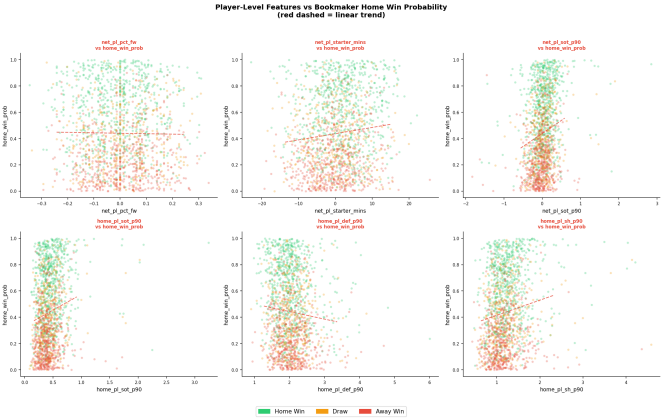
Each histogram shows the distribution of one feature split by match outcome (H/D/A).

Less overlap between the three coloured distributions ⇒ more discriminative feature.

Bookmaker probability features show the clearest separation.

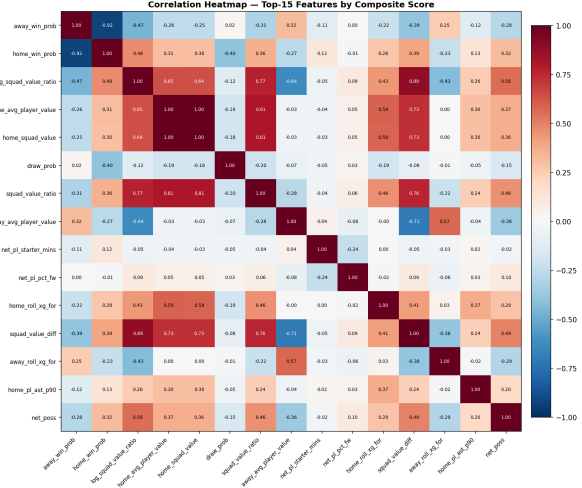
Player-level features add a small but consistent nudge on top of the bookmaker signal.

Bookmaker Odds vs Player-Level Features



The bookmaker's implied probability and the player rolling stats **mostly disagree** — wide scatter, no tight trend.

Correlation Heatmap: Top 15 Features



Red zones = high correlation ⇒ redundant (e.g. `home_win_prob` and `away_win_prob`).

Player features show near-white rows and columns ⇒ barely correlated with anything else.

This confirms they bring **genuinely new information** rather than repeating the bookmaker signal.

Linear Baseline — Setup

Model setup

Algorithm: multinomial logistic regression (softmax output)

Library: `sklearn.linear_model.LogisticRegression`

Hyperparameters: C=1.0 (default L2), `solver=lbfgs`, `max_iter=1000`

Scaling: `StandardScaler` fit on *train only*

Protocol

Split: train 2018–2022 / test 2023/24

Common subset: 593 matches with complete data across all 3 CSVs

Understat excluded from features (used as external baseline only)

3 regimes: team (29) / +FBref (50) / +player (76)

Why exclude Understat from the features?

Including the baseline as a model input would be circular – we'd partly be measuring what Understat already knows, not what our engineered features add.

Linear Baseline — Results

Test results on the held-out 2023/24 season (168 matches).

Log-loss by regime

Regime	Log-loss
A: team (29)	**1.06**
B: +FBref (50)	1.14
C: +player (76)	**1.22**
Uniform reference	1.099
Understat baseline	**0.910**

Brier (A): 0.62 · Acc. (A): 49.2%

Log-loss *increases* with more features – the central diagnostic of the project.

Linear Baseline — Limits & Diagnostic

Why did adding features hurt? Two compounding problems explain it.

1. Multicollinearity

Team-level and player-level features measure overlapping things (a team's xG is essentially a function of its players' xG). Linear models cannot decide where to attribute the weight – the optimisation has multiple equivalent solutions, leading to unstable coefficients.

2. Curse of dimensionality

Only 417 training matches against up to 76 features – a samples-to-features ratio of $\sim$5.5. Rule of thumb: linear models need 10–20$\times$ more samples than features. Below that, they **overfit the noise** rather than learn the signal.

This negative result is what motivated the tree models

Tree-based models handle correlated features natively – each split uses only one feature at a time, so they're *locally low-dimensional*. **Tree models flip the trend** (RF: 1.26 $\rightarrow$ 1.06 $\rightarrow$ 1.05). Same data, same protocol, opposite outcome – which Samuel will detail next.

Lesson: *model class matters more than feature count. The player-level signal exists – but linear models cannot surface it.*

Modeling — Hyperparameters & Tuning

Every model is tuned with `RandomizedSearchCV` (random configs drawn from each grid below), scored by **log-loss** (`neg_log_loss`), not accuracy. Cross-validation uses `TimeSeriesSplit` **(5 folds)** so the validation fold is always later in time than the training fold — data is never shuffled across the time boundary. Draw imbalance is handled with **balanced class weights** (RF) / **sample weights** (XGB); the best estimator is then **isotonic-calibrated** (`CalibratedClassifierCV`, temporal inner split) before the 50/50 RF + XGB ensemble.

Random Forest — search grid

`n_estimators`	200 / 400 / 600 / 800
`max_depth`	3 / 5 / 7 / 10 / None
`min_samples_leaf`	1 / 2 / 4 / 8
`min_samples_split`	2 / 5 / 10
`max_features`	sqrt / log2 / 0.5

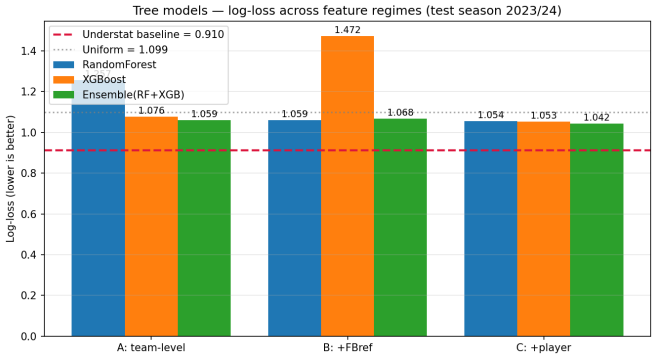
XGBoost — search grid

`n_estimators`	200 / 400 / 600 / 800
`max_depth`	3 / 4 / 5 / 6 / 8
`learning_rate`	0.01 / 0.03 / 0.05 / 0.1
`subsample`	0.6 / 0.8 / 1.0
`colsample_bytree`	0.6 / 0.8 / 1.0
`min_child_weight`	1 / 3 / 5 `reg_lambda` 0.5–5

Choosing k for KNN (same protocol)

Grid $k \in \{5, 10, 15, 20, 30, 50\} \times$ {uniform, distance} $\times$ {Manhattan, Euclidean}, scored by log-loss under `TimeSeriesSplit(5)`. Floor k=5 (smaller gives degenerate probabilities); ceiling k=50 (smallest CV fold is $\sim$75 rows). All three regimes independently picked k=50, **distance-weighted**.

Modeling — Results



Test log-loss (season 2023/24)

	A	B	C
RF	1.26	1.06	1.05
XGB	1.08	1.47	1.05
Ensemble	1.06	1.07	**1.04**
LR (ref)	1.06	1.14	1.22

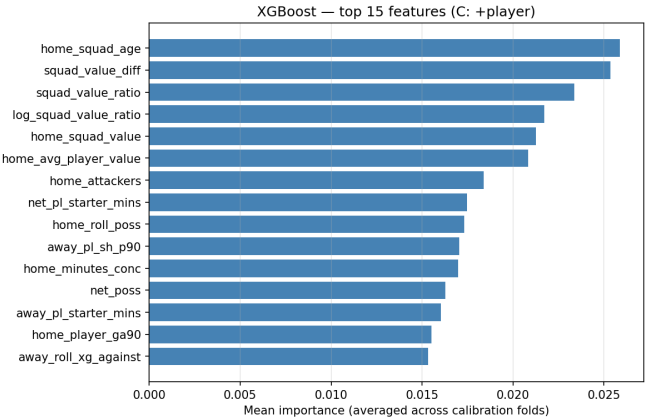
A=*team*, **B**=*+FBref*, **C**=*+player*

Accuracy in context (3-class):

Random guess	33%
Always home	43%
Best model	50%
Bookmaker	**56%**

No model clears the bookmaker baseline (0.91 log-loss). Bookmakers have real-time info (injuries, team news) that rolling historical stats cannot capture.

Modeling — Feature Importance (XGBoost, full regime)



How to read: mean importance over the calibration folds for the best XGBoost on the full player regime.

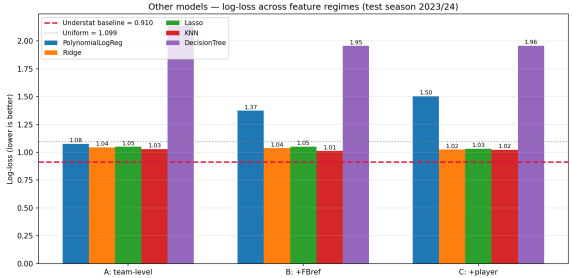
Team-level and squad-level features (squad age, squad value, market-value ratios) sit at the top, as expected.

Player-level features also enter the top ranks, especially `net_pl_starter_mins`, `away_pl_starter_mins`, They are not just noise the model tolerates, but signals it actively uses.

This is consistent with the feature-selection results.

Modeling — Other Models

Same protocol, same temporal holdout, same 3 regimes — four more model families, to test whether the algorithm matters at our sample size.



Test log-loss (season 2023/24)

	A	B	C
Polynomial	1.08	1.37	1.50
Ridge (L2)	1.04	1.04	**1.02**
Lasso (L1)	1.05	1.05	1.03
KNN	1.03	1.01	1.02
Decision Tree	2.13	1.96	1.96
RF+XGB ens.	1.06	1.07	1.04

A=*team*, **B**=*+FBref*, **C**=*+player*

Six algorithms cluster within 0.03 log-loss — we've hit the *information ceiling* of post-match aggregate stats. Closing the gap to Understat (0.91) needs richer features, not a different model.

Per-model takeaways: Polynomial overfits as features grow (d=2 explodes to 400^{+} inputs); **Ridge/Lasso** match each other — regularisation type barely matters here; **KNN** ties the tree ensemble (small dataset → simple wins); **Decision Tree** is worst in every regime — confirming the value of bagging/boosting.

Why this differs from Logistic Regression: *LR worsened as features were added (1.06 → 1.14 → 1.22) under multicollinearity and the curse of dimensionality; trees and regularised / instance-based models instead hold or improve with the extra player features — they handle correlated inputs natively.*

Key Findings

1. Player-level rolling statistics add real predictive signal beyond team form and bookmaker odds.
2. Feature selection backs that up: `net_pl_starter_mins` and `net_pl_pct_fw` rank highly across all four methods, and RFECV keeps 10 player features in its final subset.
3. Tree models give the clearest proof of the player signal: as we move from team-only features to the full player regime, every tree model improves, while logistic regression gets worse under the extra correlation and dimensionality.
4. The best tree result is the RF + XGBoost ensemble on the full player regime at **1.04** log-loss and 50.6% accuracy, so ensembling still extracts a small final gain.
5. KNN is a useful surprise: with k=50 it ties the stronger models (**1.01** in regime B, 1.02 in regime C), suggesting that averaging similar past match states works well when the sample is modest and noisy.
6. The full pipeline is fully reproducible with only public data and closes about one-third of the gap from uniform guessing to Understat's 0.91 benchmark.